```
// 06-string.js

// String - sequence of characters represented using '', "", ``
// String literal, Object literal
// String Literal type is primitive type
// create a variable, let companyName  = 'Testleaf'
// let firstName = 'TestLeaf' - both points to same memory location -
ref100,
// as both have the same value

// let lastName ='testLeaf' - it will have different memory location -
ref200,
// as it has different value
// String literal is light weight and fast and only the reference
(address ref100, ref200)
//will be stored

// It will have stack memory & heap memory
// String literal will be stored in the stack memory

// Object literal
// Object type
// let companyName1 = new string('Testleaf') // each value refer to
separate memory location
// for object literal, we need to use new keyword
// object literal, dont used in mostlyy in automation, string literal is
used
// let firstName1 = new string('Testleaf')

// object literal's address will be stord in stack memory and
// value of the object literal - 'Testleaf' will be stored in the heap
memory

// string literal - assigning value directly to the variable
// String object - create a new object using the new string() constructor

// String literal -> compare both value & datatype
let companyName = 'testleaf' //memory location 1000
let firstName = 'testleaf' // memory location 1000 (same memory so unique
data)
let firstName1 = 'Testleaf' // memory loc 2000 (different data)

console.log(companyName === firstName);
console.log(companyName === firstName1);

// String Object
// Compare object reference

let companyName1 = new String('testleaf'); // memory loc 104
let firstName2 = new String('testleaf'); // memory loc 105

console.log(companyName1 === firstName2);

// Escape sequence
let testEsc = 'it\'s a \n regression \t testing'
// ' -> single quotes not accepted, so used backslash
// \n -> to add new line use \n
// \t -> to add tab space
```

```
console.log(testEsc);

// Concatenation
let testCase = 'create a new lead'
let testCaseId = 123

//concatenate using + operator
let result =  testCase + " - " + testCaseId
console.log(testCase.concat(testCaseId.toString()))

// Modern JS - uses template literal to create string using backticks and
${}
let x = 10

let output = `There are ${x} testcases`
console.log(output)

/*
true
false
false
it's a
 regression      testing
create a new lead123
There are 10 testcases
*/
```